



University of
Nottingham

UK | CHINA | MALAYSIA

Developing Maintainable Software

COMP2013 – Lecture 01

Marjahan Begum and Max L. Wilson

Attendance





Dr Marjahan Begum – Assistant Professor

- PhD Computer Science Education from University of Nottingham (back in the days)
- Learning scientist interested in Computer Science
- Psychology of Computational context
- Women in Computing
- Human aspect of software engineering (software teams)
- How bugs are resolved in industry





Dr Max L. Wilson – Associate Professor

- **PhD:** Human-Computer Interaction
- **Research:** Brain-Computer Interaction
- **Leads:** Brain Data Group
- Taught COMP1003 for 10 years
- Director of Undergraduate Studies





Our Teaching Associates



Damla Kilic



Jwan



Liz Felton



Krishika



Today's Lecture Slot

**Introduction to
the module**

30mins

**Interview with
Eva Regalado
Kardelen Yagiz**

**Industry
Experience**

30mins

**Module
Plan**

20mins



Aims of COMP2013



Key Related Modules

Y1

**Introduction to
Software
Engineering
(1003)**

**Programming
and Paradigms
(1009)**

**Databases and
Interfaces
(1004)**

Y2

**Developing
Maintainable
Software
(2013)**

**Software
Engineering
Group Project
(2002)**

**Software
Specifications
(2010)**

Y3/4

**Software in
Society
(3013)**

**Software
Engineering
Management
(4024)**

**Distributed
Systems
(2014)**



Overall learning objectives

- That software maintenance is central to software development
 - **Maintainable software** and **software maintenance** is essential to developing large codebases
 - Esp where code is written and updated by multiple people
 - Software can only be maintained if it is maintainable
- To ensure you can write maintainable code
 - So that you understand that software maintenance is not an afterthought in the software process models and lifecycle
- To prepare you for real-world software engineering experiences



This is Most Software Engineering

- It is rare that you are involved in single SE cycle from the start
- In COMP1003, you were told:
 - Software Maintenance is the longest phase of any project
 - Software Maintenance is always another loop of the spiral model
- It is more likely that:
 - you will join a team and be asked to maintain software



This is Software Maintenance

- In most SE jobs, you will:
 - Join a team working on existing software
 - Be assigned a ticket to resolve
 - Expected to comprehend one part of a big project
 - Expected to propose a fix and get feedback
 - Expected to fix it, test it, document it, and merge request
 - Expected to improve your fix based on a code review
 - Expected to leave the software in a better state than you found it
- **This is what you will do in this module**



From Agile to DevOps

- In COMP1003, you learned about working in an agile team
 - Using agile versions of SE methods
 - BEING agile about being agile
- We taught you that it involves automated testing, CI/CD pipelines, etc
 - And that teams should work to improve these things too
 - Being Agile about Agile tools
- This is DevOps!
 - COMP2013 will teach you about DevOps



Working with BIGGER software

- In Y1, you mostly worked with scripts and small projects
- COMP2013 is about ...
 - How to approach an existing large mass/mess of code!
 - The tools you need to maintain this code "collaboratively"
- All this is very practical advice that you will find useful in the real world
- It should give good foundation for COMP2002 (the group project)



Bigger Software Projects

- We'll be working with bigger projects in Java
 - This builds on top of COMP1003 and COMP1009
- Projects have to be big enough so that:
 - We find it challenging to understand
 - They have enough complexity to refactor them
 - They have scope for new features
 - They can conform to different design patterns, coding principles and a good architecture
- COMP2013 includes the remit to teach more Desktop GUIs



COMP2013 Assumes you learned

- Programming in Java:
 - Classes and interfaces
 - Objects and inheritance
 - Java collections
 - Handling exceptions
 - Basic GUI
- Software Engineering
 - Reqs & Specs
 - UML and design
 - Software testing
 - Basic agile concepts

- Tools





**What do you think
developing maintainable
software is about?**



What are the relationships?

**Maintainable
Software (Qualities)**

**Software
Maintenance**



Maintainable software

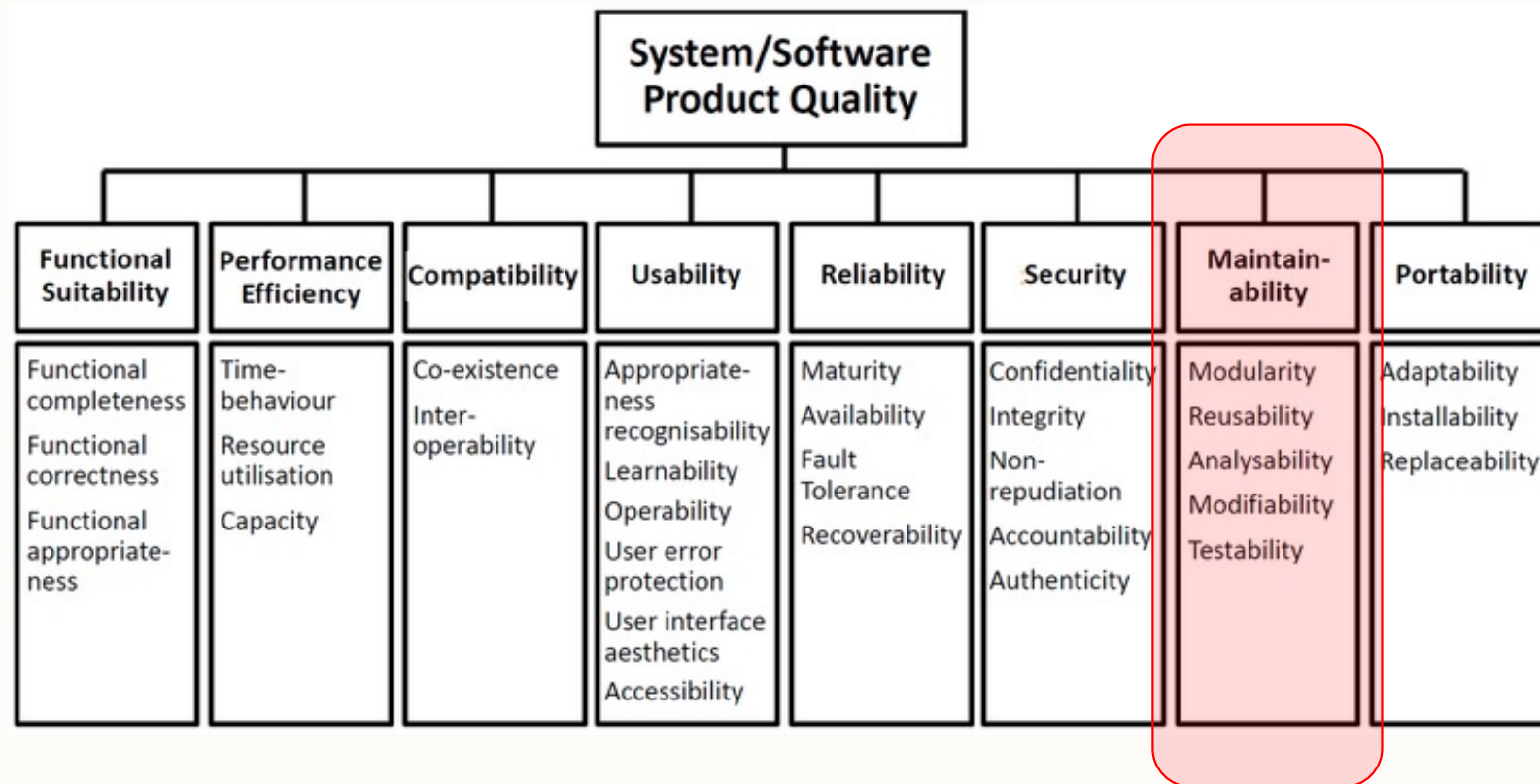
Software Quality – principles and guidelines – wish list



Developing Maintainable Software

Important !!!

■ ISO 25010





Developing Maintainable Software

Home / ISO 25000 STANDARDS / ISO 25010



Maintainability

Important !!!

This characteristic represents the degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements. This characteristic is composed of the following sub-characteristics:

- **Modularity** - Degree to which a system or computer program is composed of discrete components such that a change to one component has minimal impact on other components.
- **Reusability** - Degree to which an asset can be used in more than one system, or in building other assets.
- **Analysability** - Degree of effectiveness and efficiency with which it is possible to assess the impact on a product or system of an intended change to one or more of its parts, or to diagnose a product for deficiencies or causes of failures, or to identify parts to be modified.
- **Modifiability** - Degree to which a product or system can be effectively and efficiently modified without introducing defects or degrading existing product quality.
- **Testability** - Degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met.

<https://iso25000.com/index.php/en/iso-25000-standards/iso-25010/57-maintainability>



Maintainability

- **Modularity**

- Discrete components
- Change to one component have minimal impact on others
 - Low ripple effect

- **Reusability**

- The one bit of code is reusable in different places

- **Analysability**

- Effectively and Efficiently assess impact of individual component



Maintainability

- **Modifiability**

- Effectively and efficiently modify code
 - Without defects/degrading product quality

- **Testability**

- Effectively and efficiently write test criteria
- Test can be performed to evaluate software qualities



Developing Maintainable Software

- Three principles for developing maintainable software:
 - Maintainability benefits most from adhering to simple guidelines
 - e.g. never write methods that have more than 15 lines of code
 - Maintainability is not an afterthought, but should be addressed from the very beginning of a development project
 - but also applies to every fix you make during maintenance
 - Some violations are worse than others; the more a software system complies with the guidelines, the more maintainable it is



Developing Maintainable Software

- Some simple guidelines to develop maintainable software:
 1. Write short units (constructors/methods) of code
 2. Write simple units of code
 3. Write code once
 4. Keep unit interfaces small
 5. Separate concerns in modules (classes)
 6. Couple architecture components loosely
 7. Keep your codebase small
 8. Automate your development pipeline and tests
 9. Write clean code



Visser et al (2016)



A Poem

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.^[a]
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right now*.^[b]
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea – let's do more of those!



Software Maintenance

Software Quality – checking of it the maintainability



Software Maintenance

- What does it involve?
 - "Modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a modified environment."
- But we also need to consider how we can reduce the effort of maintenance!
 - By making sure software is easy to maintain
 - By improving software with every change, rather than making it worse

IEEE Standard for Software Maintenance:
<https://ieeexplore.ieee.org/document/257623>



Initial Development vs. Maintenance

- It's important to learn about maintenance
- It's important to build maintainable software

Original development cost	Typical annual maintenance costs	Estimated maintenance costs after 5 years
£50,000	£7,500–£12,500	£37,500–£62,500
£100,000	£15,000–£25,000	£75,000–£125,000
£250,000	£37,500–£62,500	£187,500–£312,500
£500,000	£75,000–£125,000	£375,000–£625,000

<https://www.pulsion.co.uk/blog/software-maintenance-cost-vs-development-cost/>



Resources Spend on Initial Development vs. Maintenance

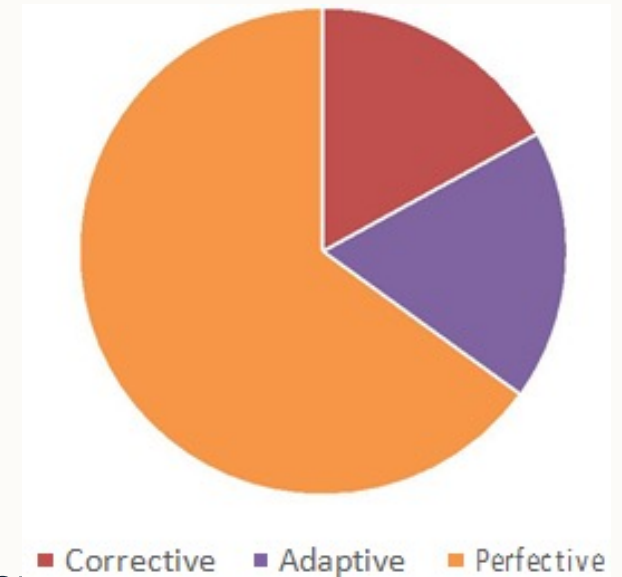




3 Main Categories of Maintenance

(or 4, depending on authors)

- Corrective Maintenance
 - Finding and fixing errors in the system
 - Example: Fixing bugs
- Adaptive Maintenance
 - Adapting to changes in the environment
 - Example: Updating for new version of windows
- Perfective + Preventive Maintenance
 - Stakeholders have new or changed requirements
 - Ways are identified to increase quality and prevent future problems





Actually there can be lots more!

- Maintenance vs Evolution?
- Maintenance:
 - Preserving software in a working state
- Evolution:
 - Improving software (e.g changing requirement)
- What we will learn will largely be applicable to both

Tripathy and Naik (2015)

TABLE 2.1 Evidence-Based 12 Mutually Exclusive Maintenance Types

Types of Maintenance	Definitions
Training	This means training the stakeholders about the implementation of the system.
Consultive	In this type, cost and length of time are estimated for maintenance work, personnel run a help desk, customers are assisted to prepare maintenance work requests, and personnel make expert knowledge about the available resources and the system to others in the organization to improve efficiency.
Evaluative	In this type, common activities include reviewing the program code and documentations, examining the ripple effect of a proposed change, designing and executing tests, examining the programming support provided by the operating system, and finding the required data and debugging.
Reformative	Ordinary activities in this type improve the readability of the documentation, make the documentation consistent with other changes in the system, prepare training materials, and add entries to a data dictionary.
Update	Ordinary activities in this type are substituting out-of-date documentation with up-to-date documentation, making semi-formal, say, in UML to document current program code, and updating the documentation with test plans.
Groomative	Ordinary activities in this type are substituting components and algorithms with more efficient and simpler ones, modifying the conventions for naming data, changing access authorizations, compiling source code, and doing backups.
Preventive	Ordinary activities in this type perform changes to enhance maintainability and establish a base for making a future transition to an emerging technology.
Performance	Activities in performance type produce results that impact the user. Those activities improve system up time and replace components and algorithms with faster ones.
Adaptive	Ordinary activities in this type port the software to a different execution platform and increase the utilization of COTS components.
Reductive	Ordinary activities in this type drop some data generated for the customer, decreasing the amount of data input to the system and decreasing the amount of data produced by the system.
Corrective	Ordinary activities in this type are correcting identified bugs, adding defensive programming strategies and modifying the ways exceptions are handled.
Enhance	Ordinary activities in this type are adding and modifying business rules to enhance the system's functionality available to the customer and adding new data flows into or out of the software.



Actually there can be lots more!

30 TAXONOMY OF SOFTWARE MAINTENANCE AND EVOLUTION

TABLE 2.1 Evidence-Based 12 Mutually Exclusive Maintenance Types

Types of Maintenance	Definitions
Training	This means training the stakeholders about the implementation of the system.
Consultive	In this type, cost and length of time are estimated for maintenance work, personnel run a help desk, customers are assisted to prepare maintenance work requests, and personnel make expert knowledge about the available resources and the system to others in the organization to improve efficiency.
Evaluative	In this type, common activities include reviewing the program code and documentations, examining the ripple effect of a proposed change, designing and executing tests, examining the programming support provided by the operating system, and finding the required data and debugging.
Reformative	Ordinary activities in this type improve the readability of the documentation, make the documentation consistent with other changes in the system, prepare training materials, and



	resources and the system to others in the organization to improve efficiency.
Evaluative	In this type, common activities include reviewing the program code and documentations, examining the ripple effect of a proposed change, designing and executing tests, examining the programming support provided by the operating system, and finding the required data and debugging.
Reformative	Ordinary activities in this type improve the readability of the documentation, make the documentation consistent with other changes in the system, prepare training materials, and add entries to a data dictionary.
Update	Ordinary activities in this type are substituting out-of-date documentation with up-to-date documentation, making semi-formal, say, in UML to document current program code, and updating the documentation with test plans.
Groomative	Ordinary activities in this type are substituting components and algorithms with more efficient and simpler ones, modifying the conventions for naming data, changing access authorizations, compiling source code, and doing backups.
Preventive	Ordinary activities in this type perform changes to enhance maintainability and establish a base for making a future transition to an emerging technology.
Performance	Activities in performance type produce results that impact the user. Those activities improve system up time and replace components and algorithms with faster ones.
Adaptive	Ordinary activities in this type port the software to a different



Groomative	semi-formal, say, in UML to document current program code, and updating the documentation with test plans. Ordinary activities in this type are substituting components and algorithms with more efficient and simpler ones, modifying the conventions for naming data, changing access authorizations, compiling source code, and doing backups.
Preventive	Ordinary activities in this type perform changes to enhance maintainability and establish a base for making a future transition to an emerging technology.
Performance	Activities in performance type produce results that impact the user. Those activities improve system up time and replace components and algorithms with faster ones.
Adaptive	Ordinary activities in this type port the software to a different execution platform and increase the utilization of COTS components.
Reductive	Ordinary activities in this type drop some data generated for the customer, decreasing the amount of data input to the system and decreasing the amount of data produced by the system.
Corrective	Ordinary activities in this type are correcting identified bugs, adding defensive programming strategies and modifying the ways exceptions are handled.
Enhancive	Ordinary activities in this type are adding and modifying business rules to enhance the system's functionality available to the customer and adding new data flows into or out of the software.



What is actually done in software maintenance?



What is involved in Software Maintenance?

- Understanding the client (need to listen and translate into requirements)
- Understanding the existing code (often harder than you think)
- Refactoring the existing code (apply the guidelines for developing maintainable code)
- Extending the existing code (adding functionality)
- Working as a team
- Managing client expectations (deliver what the client wants to a high standard in time)
- Managing the maintenance process (same as development; need to be organised)



Maintenance as a Murder Mystery!

- Fixing a problem requires in depth knowledge of the crime scene, some specialist tools, and detective work!
 - Where did the crime take place?
 - Who committed the crime?
 - (not a witch hunt – but useful to know to try to locate related bugs)
 - Who and what else was involved?
 - Part detective, part programmer



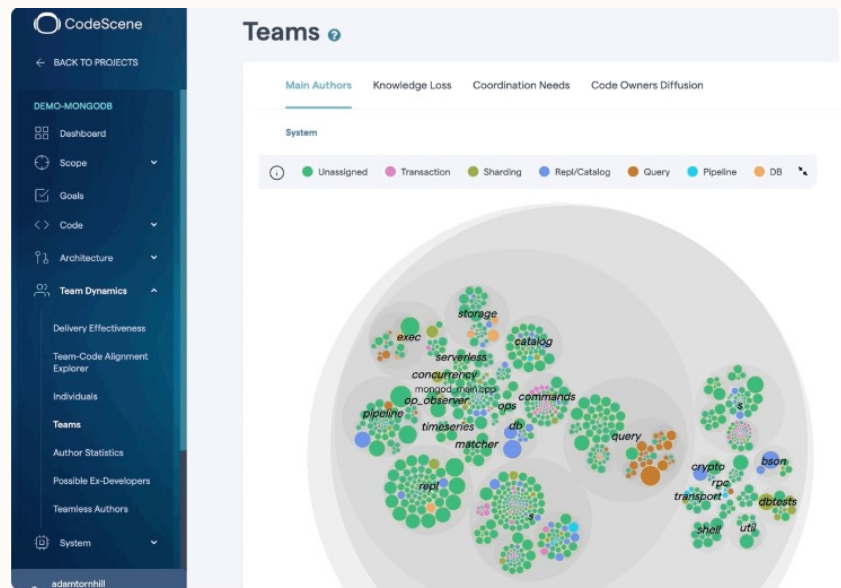


Maintenance as a Murder Mystery!

- Some tools take this analogy a step further
 - e.g. CodeScene (<https://codescene.com/>)
 - Identifies patterns in the evolution of your code
 - Software forensics

<https://www.youtube.com/watch?v=a5sZVjlxg8E>

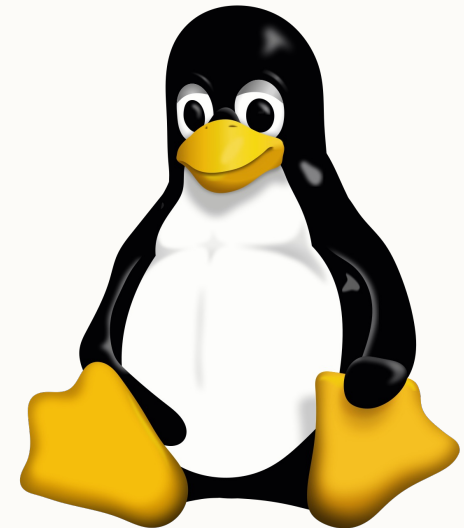
<https://www.youtube.com/watch?v=64cIvEIniu4>





How big is big? How complex is complex?

- Example: Linux Kernel
 - How many lines of code?
 - 15 million lines of code (204.5 for a full Linux distribution)
 - How many patches per release?
 - Approx. 10,000 patches in each release
 - Releases every 2-3 month
 - How many developers?
 - Each release by 1,000s of developers
 - 200 corporations

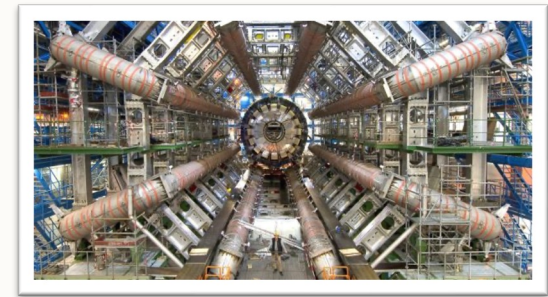
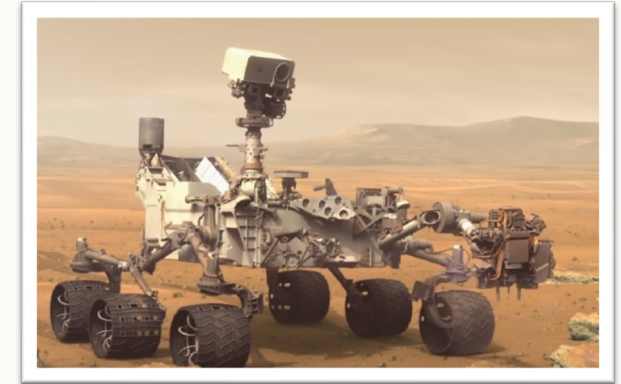


<http://www.linuxfoundation.org>



How big is big?

- Curiosity: 2.5 million lines
- CERN: 3 million lines (+50 million in projects)
 - Large hadron collider (particle accelerator)
- Windows: 50 million lines
- The code that guided the Apollo 11 module in 1969: 100,000 lines
- Of course, it is not just lines of code
 - 4000 people worked on the code base of Windows 10





Software Acquisition

- It's common for software to be acquired by bigger companies
- You have to bring the software to company standards / expectations
- The plan is to integrate the software
 - so it is part of the same software
- What's the first step?



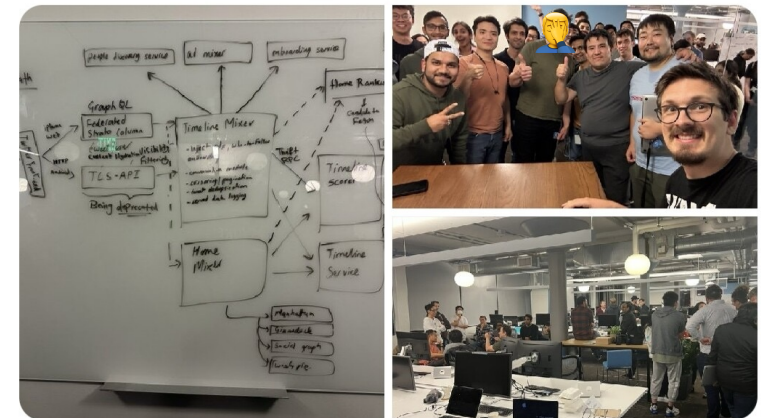
Here it is: Twitter engineers were told today to *print out* their last 30 to 60 days of code, so they could show it to New CEO himself.

Then they were told wait, no, actually, please shred all that code you just printed out.

Subscribe to read ➔

platformer.news/p/elon-takes-o...

Just leaving Twitter HQ code review



4:28 AM · 19 Nov 22 · [Twitter for iPhone](#)

27.1K Retweets **11.4K** Quote Tweets **330K** Likes



Twitter overhaul

Our “algorithm” is overly complex & not fully understood internally. People will discover many silly things , but we’ll patch issues as soon as they’re found!

We’re developing a simplified approach to serve more compelling tweets, but it’s still a work in progress. That’ll also be open source.

Providing code transparency will be incredibly embarrassing at first, but it should lead to rapid improvement in recommendation quality. Most importantly, we hope to earn your trust.

5:16 PM · Mar 17, 2023 · **9.8M** Views

Btw, I’d like to apologize for Twitter being super slow in many countries. App is doing >1000 poorly batched RPCs just to render a home timeline!

1:00 PM · Nov 13, 2022 · Twitter for iPhone



How do you handle the new code?

- Some suggestions from last year's cohort
 - Try to understand the code
 - How do you do it?
 - Look for the documentation
 - Let's hope the person who has written the code base has done the documentation!
 - Talk to team members
 - Scary thought? Need to learn to communicate
 - This is one of the key objectives of the group project!



What is involved in Software Maintenance?

- Understanding the client
- Understanding the code
- Refactoring the code
- Extending the code
- Working as a team
- Managing client expectations
- Managing maintenance process



What is involved in Software Maintenance?

- Understanding the client
- **Understanding the code (1a/b)**
 - Refactoring the code
 - **Extending the code (2)**
- Working as a team
- Managing client expectations
- **Managing maintenance process (3)**

Important !!!



1a. Making Sense of System Structure

- There could be hundreds or thousands of source code files in a project
- Program comprehension accounts for 50% of the total effort expended throughout the life cycle of a software system (Tripathy and Naik 2015).
- How to make sense of them?
 - It makes sense to look at relationships between classes
 - To do this we can use visualisation techniques
 - Understanding OOP is critical here





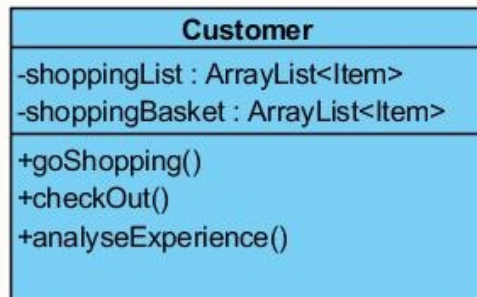
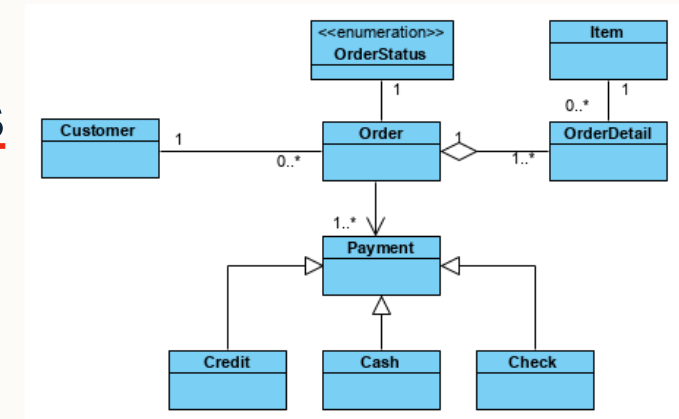
1a. Making Sense of System Structure

■ Class Diagrams

- Show a set of classes and their relationships
- Addresses static design view of a system

■ Classes

- Blueprints (templates) for objects
- Contain data/information and perform operations

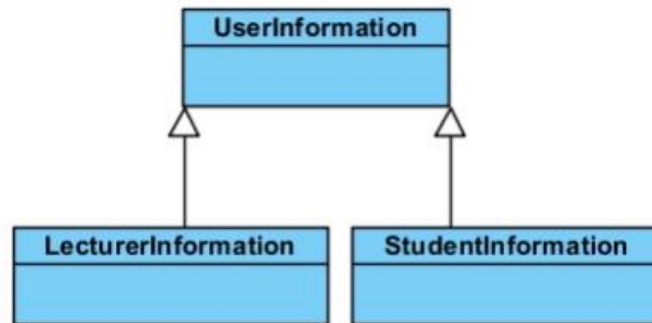


Test yourself:
What is in the three compartments?



1a. Making Sense of System Structure

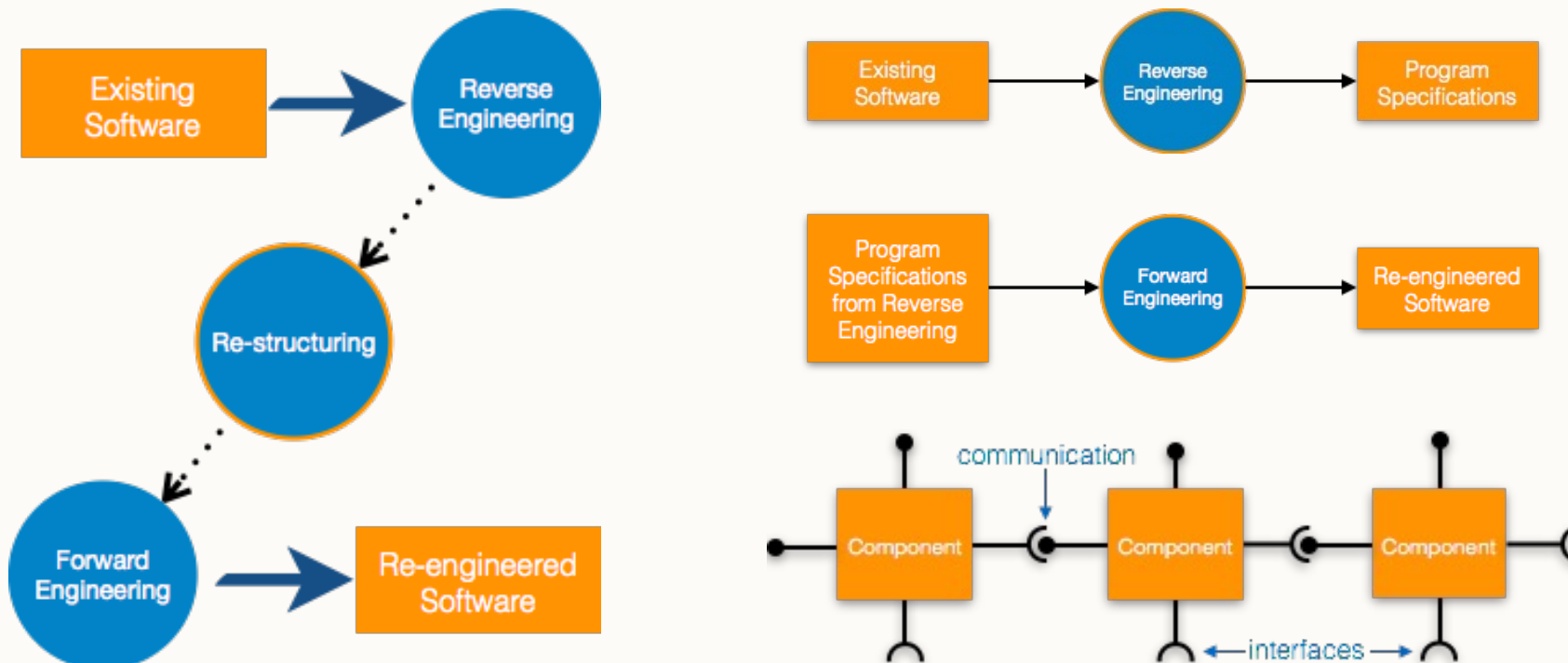
- Class diagrams offer a great way of showing inheritance and polymorphism



Test yourself: does this show inheritance or polymorphism?



1a. Reverse Engineering



https://sceweb.sce.uhcl.edu/helm/WEBPAGES-SoftwareEngineering/myfiles/TableContents/Module-13/software_maintenance_overview.html



1a. Reverse Engineering

- Transform these classes into a simple class diagram and do some maintenance

```
1 package ZooSystem;  
2  
3 public class Piranha extends Fish{  
4 }
```

```
1 package ZooSystem;  
2  
3 public abstract class Bird extends Animal{  
4 }
```

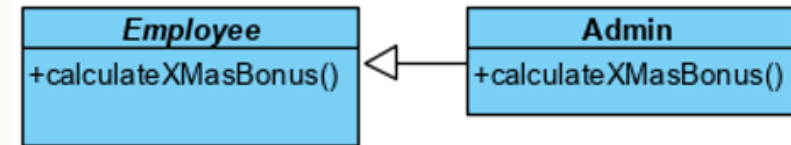
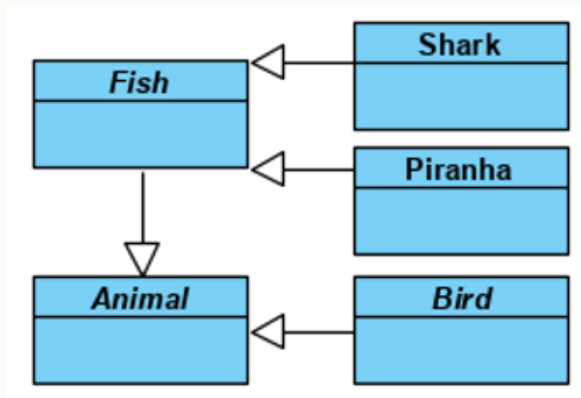
```
1 package ZooSystem;  
2  
3 public class Shark extends Fish{  
4 }
```

```
1 package ZooSystem;  
2  
3 public class Admin extends Employee{  
4  
5     @Override  
6     public int calculateChristmasBonus(){  
7         int bonus=(int)((double)getSalary()*0.08);  
8         return bonus;  
9     }  
10  
11     public Admin(String name){  
12         super();  
13         setEmployeeName(name);  
14     }  
15 }
```



1a. Reverse Engineering

- Simple class diagram (only including relevant methods)





1a. UML in Practice

- Class diagrams can help sometimes (but not always)
- They summarise the content of classes and relationships between them in the simplest possible way
- Does it make sense to build a detailed *class diagram* for an entire BIG project?
 - There are other methods to do this and other UML techniques to look at software at a higher level (e.g. deployment diagrams)
- Code can be generated directly from UML for maximum consistency



1a. COMP2013 </> UML in Practice

- Caution! This is still a relatively small number of classes
- Use the diagrams sensibly, where they can help
- In this case using a high-level class diagram would be advisable



GenMyModel
@GenMyModel

Follow

Wow! User rajiv's #UML class diagram is one of the biggest we've seen! Impressive!

goo.gl/ll5dIY





1b. Understanding the code itself

- As well as understanding the structure, we need to understand what the code actually does
- We need to understand the code itself to make the right decisions for producing robust and maintainable code



1b. Understanding the code itself

- Let's consider adding a method to set an employee phone number
 - Which class should it go in?
 - In Admin, or the abstract class Employee?
 - Should it be public or private?
 - What variable will you use?
 - What type?
- What if another subtype of Employee already has a telephone method?
 - And what if one doesn't have a fixed telephone (**mobile number is share by a number of employees?**)

```
1 package ZooSystem;
2
3 public class Admin extends Employee{
4
5     @Override
6     public int calculateChristmasBonus(){
7         int bonus=(int)((double)getSalary()*0.08);
8         return bonus;
9     }
10
11     public Admin(String name){
12         super();
13         setEmployeeName(name);
14     }
15 }
```



1b. Following conventions vs being smart

- What does this C snippet do? Could you extend it easily?

```
float ████████( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y = number;
    i = * ( long * ) &y;
    i = 0x5f3759df - ( i >> 1 );
    y = * ( float * ) &i;
    y = y * ( threehalfs - ( x2 * y * y ) );
    // y = y * ( threehalfs - ( x2 * y * y ) );

    return y;
}
```

- No comments
- Poor variable names
- Obscure coding
- Redundant commenting out



Actually, it's Quake3's Fast InvSqrt()



1b. Following conventions vs being smart

- Actually, there were some comments – *how helpful are these?*

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

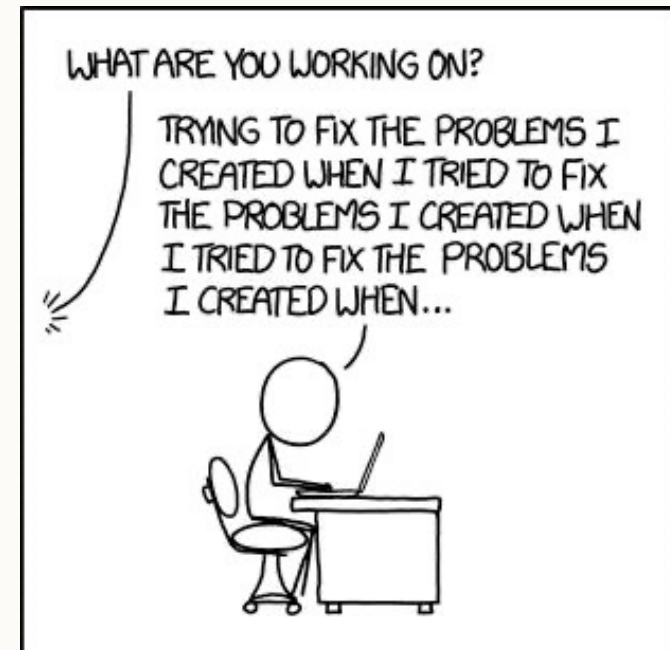
    x2 = number * 0.5F;
    y = number;
    i = * ( long * ) &y;                // evil floating point bit level hacking
    i = 0x5f3759df - ( i >> 1 );        // what the 5?
    y = * ( float * ) &i;
    y = y * ( threehalfs - ( x2 * y * y ) ); // 1st iteration
    // y = y * ( threehalfs - ( x2 * y * y ) ); // 2nd iteration, this can be removed

    return y;
}
```



2. Extending Existing Code

- When you change existing code you risk breaking it
 - How do you know you haven't broken it?
 - What kind of testing do you need?
- Regression testing
 - A type of software testing to confirm that a recent program or code change has not adversely affected existing features
 - Running all your unit tests again





3. Software Maintenance Management

- This involves the management of people and not the management of conditional statements or linked lists
- The most under-appreciated part of our course, yet deep understanding of these processes often leads to successful future employment
- You are no longer the "Lone Coder" – from now on we encourage you to think like a pro and write your code as if someone else will need it
- You have looked at Git and version control in Y1; we will be going over it again ... and again ... and again ... until you use it by habit.
- Understanding how open source and software licensing works



3. Software Maintenance Management

- Use established processes to manage the design and implementation of changes
- Our Industrial Steering Group wants you to learn about how to actually perform agile development ... not just to know the word
 - Team meetings and a design process – e.g. Scrum
 - Team communication
 - Source code management (e.g. GIT)
 - Server-side testing (continuous integration server)
- This means talking to other people ...





What is involved in Software Maintenance?

- Understanding the client
- Understanding the code
- Refactoring the code
- Extending the code
- Working as a team
- Managing client expectations
- Managing maintenance process